

CHAPTER-1

1. INTRODUCTION

Software architecture serves as the foundation of every successful software system. Before development begins, architects and software engineers must prepare High-Level Design (HLD) and Low-Level Design (LLD) documents that define system structure, modules, communication patterns, technology stacks, databases, deployment architecture, and security considerations.

In modern enterprise environments, creating architecture blueprints is a time-consuming process requiring experienced architects. Organizations spend considerable effort analyzing requirements, selecting technologies, defining deployment models, and preparing design documentation. This manual process often results in delays, inconsistencies, and increased project costs.

Artificial Intelligence (AI) has emerged as a transformative technology capable of automating various Software Development Life Cycle (SDLC) activities. AI-based systems can analyze requirements, identify application patterns, recommend technologies, and generate design artifacts automatically.

ArchiGen AI is an intelligent architecture blueprint generation system designed to automate software architecture planning. The system analyzes project requirements and generates both High-Level Design (HLD) and Low-Level Design (LLD) blueprints automatically. It recommends suitable technologies for frontend, backend, databases, cloud infrastructure, security, and deployment environments.

The proposed system aims to support multiple enterprise domains including Healthcare, Banking Financial Services and Insurance (BFSI), E-Commerce, Education, and Government applications. By leveraging AI-driven recommendation engines and requirement analysis mechanisms, ArchiGen AI significantly reduces architecture planning time and improves SDLC efficiency.

The system serves as a bridge between business requirements and technical implementation by transforming user requirements into structured architecture blueprints. This automation helps organizations accelerate project initiation, improve consistency, and reduce dependency on architecture experts.

Software architecture defines how software components interact and communicate. Over the years, architecture styles such as Monolithic Architecture, Service-Oriented Architecture (SOA), Microservices Architecture, Event-Driven Architecture, and Cloud-Native Architecture have emerged to address different business needs.

With increasing software complexity, selecting the right architecture and technology stack has

become more challenging. Organizations need tools that can simplify architecture planning and reduce dependency on manual expertise.

ArchiGen AI addresses this challenge by combining Artificial Intelligence, Natural Language Processing (NLP), recommendation systems, and architecture pattern recognition to automate blueprint generation.

Software architecture blueprint generation is a critical but time-consuming activity in SDLC. Existing approaches depend heavily on manual effort from software architects and system engineers. Different application domains require different architecture patterns, deployment strategies, and technology stacks. Selecting suitable technologies and preparing HLD/LLD documents requires extensive expertise and significant time investment.

Therefore, there is a need for an intelligent AI-driven system capable of automatically analyzing requirements, recommending technologies, and generating architecture blueprints to improve efficiency and reduce planning effort.

The motivation behind ArchiGen AI arises from the increasing demand for rapid software delivery and efficient architecture planning. Organizations require solutions that reduce project planning effort while maintaining design quality.

By automating architecture generation, ArchiGen AI can:

- Reduce planning time
- Improve consistency
- Accelerate project initiation
- Support architecture decision-making
- Increase development productivity

CHAPTER-2

LITERATURE SURVEY AND BACKGROUND STUDY

2.1 LITERATURE SURVEY

Sl. No	Paper Title	Author(s)	Year	Contribution	Limitation
1	Software Architecture Meets Large Language Models: A Systematic Literature Review	Schmid et al.	2025	Reviews LLM usage in architecture generation	Limited industrial validation
2	Artificial Intelligence for Software Architecture: Literature Review and the Road Ahead	Bucaioni et al.	2025	Comprehensive AI architecture survey	Limited implementation focus
3	Generative AI for Software Architecture: Applications, Trends, Challenges, and Future Directions	Esposito et al.	2025	AI-based architecture generation	Emerging research area
4	How Mature is Requirements Engineering for AI-Based Systems?	Habiba et al.	2024	AI requirement engineering analysis	Limited architecture focus
5	The Use of AI in Software Engineering: A Synthetic Knowledge Synthesis	Kokol	2024	AI applications in software engineering	Broad scope
6	Formal Software Architecture Rule Learning	Schindler & Rausch	2024	Architecture rule generation using AI	Limited scalability

7	Future of Software Development with Generative AI	Sauvola et al.	2024	Generative AI in SDLC	Validation challenges
8	Generative AI for Requirements Engineering: A Systematic Review	Cheng et al.	2024	AI requirement automation	No architecture generation
9	MIRROR: Multi-Objective Refactoring Recommendation	Zhang et al.	2024	AI architecture improvement	Refactoring-focused
10	Design Principles for Generative AI Applications	Weisz et al.	2024	AI system design framework	Not architecture specific
11	Content-Centric Prototyping of Generative AI Applications	Subramonyam et al.	2024	AI-assisted software prototyping	Limited architecture support
12	Requirements Engineering for Artificial Intelligence Systems	Ahmad et al.	2023	AI requirement analysis	Limited design automation
13	Requirements Engineering for AI Systems: Mapping Study	Ahmad et al.	2023	Requirement engineering survey	No blueprint generation
14	Identifying AI Applications in Requirements Engineering	Dehn et al.	2023	AI-assisted requirement processes	Limited deployment support
15	From Monolithic to Microservice Architecture: Automated Approach	Filippone et al.	2023	Automated architecture migration	Scalability issues
16	Software Evolutionary Architecture: Automated Planning	Elsevier Research Group	2023	Architecture planning automation	Complex implementation

17	Deep Learning-Based Software Process Model Recommendation	Alshammari	2022	SDLC recommendation system	Limited architecture generation
18	Explainability of Generative AI for Code Through Scenario-Based Design	Sun et al.	2022	AI-assisted software design	Focused on code generation

Table (a):Literature Survey

Software architecture generation and Software Development Life Cycle (SDLC) automation have become important research areas due to the increasing complexity of modern software systems. Traditional architecture planning requires significant manual effort from software architects and system designers. Recent advancements in Artificial Intelligence (AI), Generative AI, Large Language Models (LLMs), Natural Language Processing (NLP), and cloud computing have created opportunities to automate architecture design, requirement analysis, and software planning processes. The proposed system, ArchiGen AI: Autonomous Architecture Blueprint Generation for Intelligent SDLC Automation, is inspired by these emerging research developments.

Schmid et al. (2025) conducted a systematic literature review on the application of Large Language Models in software architecture generation. The study analyzed how LLMs are being used for architecture decision support, architecture reconstruction, documentation generation, and design recommendation. The research highlighted the growing role of AI in software architecture while identifying challenges such as limited industrial validation, model reliability, and architecture-specific datasets. This work demonstrates the potential of AI-driven architecture generation systems.

Bucaioni et al. (2025) presented a comprehensive review of Artificial Intelligence applications in software architecture. Their study explored architecture recommendation systems, architectural decision support tools, and AI-assisted design generation frameworks. The authors concluded that AI has the potential to improve architecture planning efficiency but emphasized the need for stronger evaluation frameworks and practical implementation approaches. Their findings provide valuable insights into the future direction of AI-powered architecture automation.

Esposito et al. (2025) investigated Generative AI applications in software architecture and analyzed current trends, challenges, and future opportunities. The study found that Generative AI is increasingly being used for requirements-to-architecture transformation, architecture reconstruction, and technology recommendation. The authors also identified challenges such as

hallucination issues, lack of explainability, privacy concerns, and insufficient validation techniques. This research strongly aligns with the objectives of ArchiGen AI.

Habiba et al. (2024) focused on Requirements Engineering for AI-based systems through a systematic mapping study. The research analyzed how requirements engineering practices are evolving to support AI-driven software systems. The study highlighted the importance of automated requirement analysis and identified challenges in requirement specification, validation, and traceability. These findings are relevant for the requirement analysis component of ArchiGen AI.

Kokol (2024) presented a synthetic knowledge synthesis of AI applications in software engineering. The study reviewed recent advancements in AI-assisted software development, project management, testing, requirement engineering, and architecture design. The research emphasized that AI technologies can significantly improve software engineering productivity while reducing repetitive manual tasks. However, challenges related to trust, explainability, and reliability remain significant concerns.

Schindler and Rausch (2024) proposed a framework for learning software architecture rules using Large Language Models and inductive techniques. Their study demonstrated how AI models can automatically identify and generate architecture rules based on software design patterns. The research contributes toward automated architecture generation and supports the concept of intelligent architecture recommendation systems.

Sauvola et al. (2024) examined the future of software development with Generative AI technologies. The study discussed how AI can automate coding, testing, documentation, and software design processes. The authors predicted that Generative AI would become a major component of future SDLC automation frameworks. However, concerns regarding accuracy, governance, and ethical usage were identified as key challenges.

Cheng et al. (2024) conducted a systematic literature review on Generative AI applications in Requirements Engineering. The study analyzed over one hundred research papers and identified how GPT models and other LLMs are being used to automate requirement gathering, analysis, classification, and validation. The authors highlighted interpretability, reproducibility, and controllability as major technical challenges while emphasizing the importance of standardized evaluation frameworks.

Zhang et al. (2024) introduced MIRROR, an AI-based multi-objective refactoring recommendation framework. The system uses machine learning techniques to analyze software structures and recommend architecture improvements. The research demonstrated how AI can support software evolution and maintainability by identifying optimal refactoring strategies. However, the framework

primarily focuses on refactoring rather than complete architecture generation.

Weisz et al. (2024) proposed design principles for Generative AI applications. Their research provides guidelines for developing AI-powered software systems and emphasizes user interaction, transparency, explainability, and responsible AI usage. Although not specifically focused on software architecture generation, the study offers valuable design considerations for AI-driven software engineering systems such as ArchiGen AI.

Subramonyam et al. (2024) explored content-centric prototyping of Generative AI applications. Their study demonstrated how AI can assist software teams during prototyping, requirement analysis, and design creation. The research highlights the role of AI in accelerating software planning activities and improving collaboration between stakeholders and developers. This concept directly supports automated blueprint generation systems.

Ahmad et al. (2023) conducted a systematic mapping study on Requirements Engineering for Artificial Intelligence systems. The research analyzed current methodologies for requirement elicitation, specification, validation, and management in AI-driven applications. The authors identified several research gaps and highlighted the need for intelligent requirement analysis frameworks capable of handling complex software projects.

Dehn et al. (2023) investigated the use of Artificial Intelligence within requirements engineering processes. Their study examined how AI techniques can support requirement identification, prioritization, validation, and documentation. The results showed that AI can improve requirement management efficiency but requires further integration with software design and architecture planning tools.

Filippone et al. (2023) proposed an automated approach for migrating monolithic applications to microservice architectures. The system analyzes software structures and recommends architectural transformations based on application characteristics. The study demonstrates the importance of architecture automation and supports the development of intelligent architecture recommendation systems.

The Software Evolutionary Architecture study (2023) focused on automated planning for software architecture evolution. The framework supports architecture adaptation and functional changes throughout the software lifecycle. The research emphasizes the importance of automated decision-making and planning mechanisms in maintaining scalable software systems.

Alshammari (2022) proposed a deep learning-based software process model recommendation system. The framework uses machine learning algorithms to recommend suitable SDLC models and development processes based on project characteristics. This work demonstrates the growing role

of AI in SDLC automation and project planning activities.

Sun et al. (2022) investigated explainability in Generative AI for software development. Their research examined how AI-generated code and design recommendations can be made more transparent and understandable for developers. The study highlights the importance of explainable AI in software engineering applications.

Overall, the literature survey reveals significant advancements in AI-assisted software engineering, requirements engineering, software architecture automation, and SDLC optimization. However, most existing systems focus on individual aspects such as requirement analysis, architecture recommendation, cloud deployment planning, or code generation. There remains a research gap in developing a unified AI-driven platform capable of analyzing requirements, recommending technologies, generating High-Level Design (HLD) documents, generating Low-Level Design (LLD) blueprints, and supporting complete architecture planning. This gap motivates the development of ArchiGen AI, which aims to provide an integrated solution for autonomous architecture blueprint generation and intelligent SDLC automation.

2.2 BACKGROUND STUDY

Software architecture is a fundamental aspect of software engineering that defines the overall structure, behavior, and interactions of a software system. It serves as a blueprint for system development by outlining components, communication mechanisms, technology stacks, databases, deployment strategies, and security considerations. Traditionally, software architects manually analyze project requirements and create High-Level Design (HLD) and Low-Level Design (LLD) documents to guide the development process. However, this approach requires significant expertise, time, and effort, especially for large-scale enterprise applications.

The emergence of Artificial Intelligence (AI), Machine Learning (ML), and Natural Language Processing (NLP) has transformed many areas of software engineering. AI-powered systems are increasingly being used for requirement analysis, code generation, testing, project management, and decision support. Recent research has shown that AI can assist in identifying application patterns, recommending suitable technologies, and automating repetitive development tasks. Similarly, cloud computing, DevOps practices, containerization technologies such as Docker and Kubernetes, and microservices architectures have changed how modern software systems are designed and deployed.

Several existing tools provide support for architecture documentation and visualization, but most of them still rely on manual input from software architects. Current systems often focus on a single aspect of architecture planning, such as requirement analysis, technology recommendation, or cloud deployment design. There is limited support for generating complete architecture blueprints automatically. This creates a need for an intelligent solution that can analyze project requirements, recommend suitable technologies, and generate comprehensive HLD and LLD documents with minimal human intervention.

ArchiGen AI builds upon these advancements by integrating AI-based requirement analysis, technology recommendation, and architecture blueprint generation into a single platform. The system aims to automate architecture planning, reduce manual effort, improve consistency, and accelerate the Software Development Life Cycle (SDLC). By leveraging modern AI techniques and software engineering principles, ArchiGen AI contributes to the growing field of intelligent SDLC automation and software architecture generation.

CHAPTER-3

GAP ANALYSIS, JUSTIFICATION AND SCOPE

3.1 3GAP ANALYSIS

The literature survey and background study reveal that existing software architecture solutions provide only partial automation. Many systems focus on requirement analysis, technology recommendation, cloud architecture planning, or architecture visualization independently. Most architecture planning activities still require extensive manual effort from software architects and domain experts. Existing tools lack the ability to automatically generate both High-Level Design (HLD) and Low-Level Design (LLD) blueprints from project requirements. Additionally, many solutions are domain-specific, offer limited scalability, and do not provide a unified framework for architecture planning and SDLC automation. These limitations highlight the need for a comprehensive AI-driven architecture generation system.

Existing systems:

- Require manual architecture planning.
- Depend on expert architects.
- Lack automated HLD generation and LLD generation.
- Require multiple tools.
- Limited AI integration.

Research Gaps:

- No complete AI-driven blueprint generation.
- Lack of unified architecture automation platform.
- Limited technology recommendation engines.
- Poor SDLC integration.

3.2 JUSTIFICATION

ArchiGen AI addresses these gaps by providing a unified AI-powered platform for architecture planning and blueprint generation.

The development of ArchiGen AI is justified by the increasing complexity of modern software systems and the growing demand for faster project delivery. Organizations require efficient methods to reduce architecture planning time while maintaining design quality and consistency. By automating requirement analysis, technology selection, and blueprint generation, ArchiGen AI can significantly reduce manual effort and improve productivity. The system provides a unified platform that supports multiple enterprise domains and helps organizations accelerate software development while minimizing dependence on highly experienced architects. This makes the proposed solution both practical and valuable in modern software engineering environments.

3.3 SCOPE

The scope of ArchiGen AI includes automatic requirement analysis, technology recommendation, High-Level Design (HLD) generation, Low-Level Design (LLD) generation, and architecture blueprint visualization. The system is designed to support various enterprise domains such as Healthcare, Banking and Financial Services (BFSI), E-Commerce, Education, and Government applications. It aims to improve SDLC efficiency by providing intelligent architecture recommendations and automated documentation. In the future, the scope can be extended to include DevOps pipeline generation, cloud deployment automation, AI-assisted code generation, automated testing strategies, and continuous architecture optimization, making it a comprehensive intelligent software engineering platform.

Current Scope

- HLD generation
- LLD generation
- Technology recommendation
- Architecture visualization

Future Scope

- DevOps pipeline generation
- Cloud deployment automation
- AI code generation
- Automated testing strategy generation

CHAPTER-4

PROBLEM STATEMENT AND OBJECTIVES

4.1 PROBLEM STATEMENT

Architecture blueprint generation for enterprise applications is a time-consuming and complex process. Different project types require different planning strategies and technology selections. Existing systems depend heavily on manual effort from architects and engineers. Therefore, an AI-based automated system is needed for efficient HLD and LLD blueprint generation.

4.2 OBJECTIVES OF THE PROJECT WORK

The primary objective of ArchiGen AI: Autonomous Architecture Blueprint Generation for Intelligent SDLC Automation is to develop an intelligent AI-driven system that automates the software architecture planning process. The system aims to analyze project requirements and generate comprehensive High-Level Design (HLD) and Low-Level Design (LLD) blueprints automatically, reducing the manual effort traditionally required from software architects.

The specific objectives of the project are:

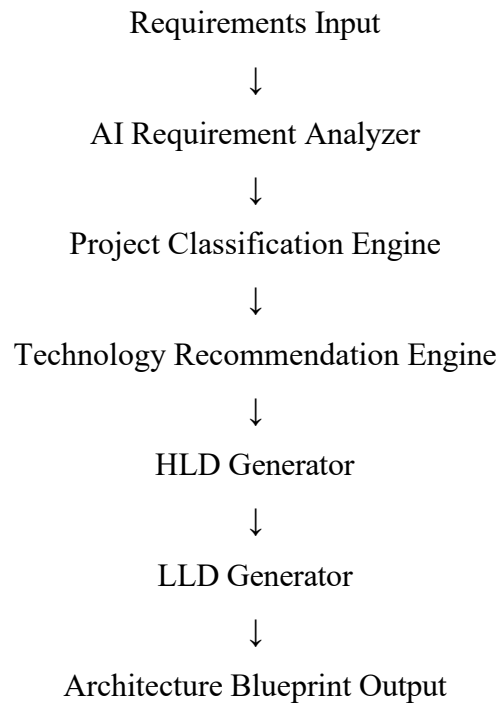
- To analyze software project requirements and identify the application domain automatically using Artificial Intelligence techniques.
- To generate High-Level Design (HLD) blueprints that provide a structured overview of system architecture, modules, deployment strategies, and component interactions.
- To generate Low-Level Design (LLD) blueprints containing detailed module specifications, database structures, API designs, and implementation-level details.
- To recommend suitable technologies for different architectural layers, including frontend, backend, databases, cloud infrastructure, security frameworks, and deployment platforms.
- To reduce the time and effort involved in software architecture planning and documentation.
- To improve consistency and accuracy in architecture design by minimizing human errors and subjective decision-making.
- To support multiple enterprise domains such as Healthcare, Banking and Financial Services (BFSI), E-Commerce, Education, and Government applications.
- To provide visual architecture blueprints and structured design documents that assist developers, architects, and project managers throughout the SDLC.
- To enhance software development productivity by automating repetitive planning activities and accelerating project initiation.
- To create a scalable and extensible platform that can support future enhancements such as automated DevOps pipeline generation, cloud deployment automation, and AI-assisted software development. Overall, ArchiGen AI aims to transform traditional software architecture planning into an intelligent, automated, and efficient process, thereby improving software quality, reducing project delays, and supporting faster delivery of enterprise applications.

CHAPTER-5

5. PROPOSED SYSTEM DESIGN / METHODOLOGY

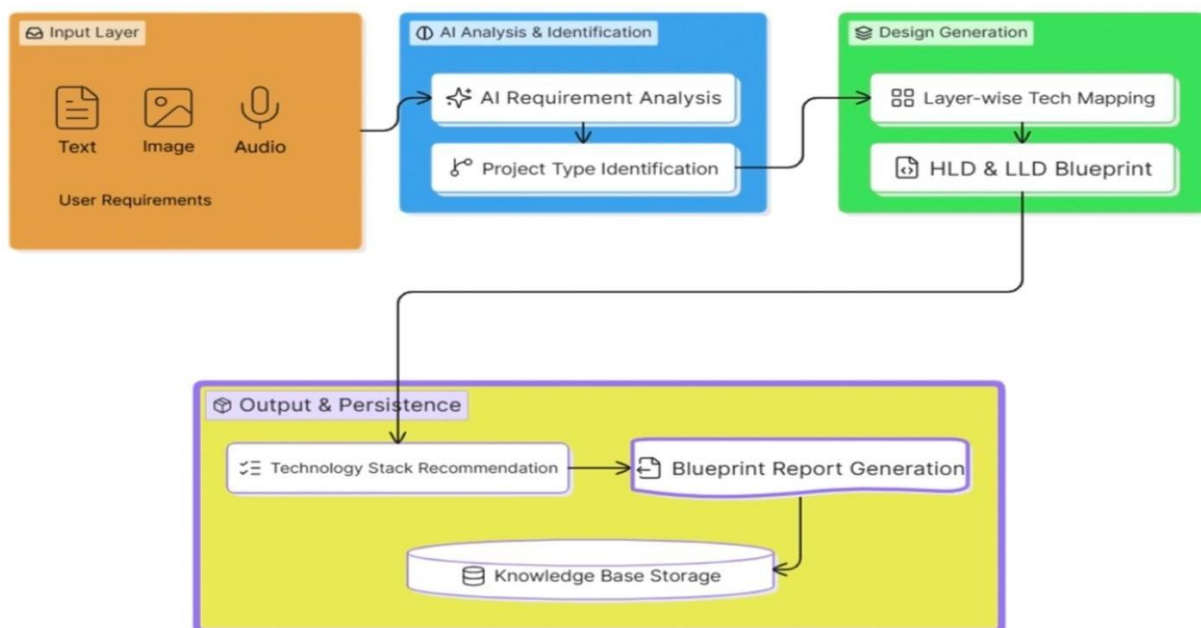
System Architecture

Flow:



Workflow Diagram

AI Blueprint Generation Architecture



fig(a):Workflow diagram

This architecture describes how raw user requirements are transformed into a structured AI system blueprint. It follows a pipeline of four stages: input, analysis, design generation, and output/persistence. Each stage progressively refines the information until a complete technical design is produced and stored.

Workflow Stages:

- **User Requirements :**

The process begins with inputs from the user, which can be in the form of text, images, or audio. These inputs capture the problem statement, goals, or constraints that the AI system must address.

- **AI Requirement Analysis :**

The system interprets the user's inputs, extracting key needs and functional expectations. This ensures the requirements are understood in technical terms.

- **Project Type Identification :**

Based on the analysis, the system determines the type of AI project (e.g., machine learning model, natural language processing system, computer vision solution). This classification guides the design choices.

- **Layer-wise Tech Mapping:**

The architecture maps each requirement to appropriate technologies across layers (data, model, interface, deployment). This ensures the right tools and frameworks are chosen.

- **HLD & LLD Blueprint :**

Two levels of design are generated:

High-Level Design (HLD): Overall system structure, components, and interactions.

Low-Level Design (LLD): Detailed specifications, algorithms, and workflows.

- **Technology Stack Recommendation :**

The system suggests the most suitable programming languages, frameworks, and platforms to implement the blueprint.

- **Blueprint Report Generation :**

A formal document is produced, summarizing the design, stack, and architecture for stakeholders.

- **Knowledge Base Storage :**

Finally, the blueprint and related insights are stored in a knowledge base for future reference, reuse, or iterative improvement.

This workflow ensures that unstructured user requirements are systematically converted into a structured, actionable AI blueprint. It bridges the gap between what a user wants and how engineers can build it, while also preserving knowledge for long-term scalability.

Modules

Requirement Analysis Module:

The Requirement Analysis Module is the first and most important component of ArchiGen AI. This module accepts project requirements from users in the form of text descriptions, documents, or specifications. Using Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques, it analyzes the requirements to identify key functionalities, business objectives, system constraints, and application characteristics. The module also determines the application domain, such as Healthcare, Banking, E-Commerce, or Education. By automatically understanding project requirements, this module reduces manual analysis effort and provides the foundation for architecture generation.

- NLP processing
- Requirement extraction
- Domain identification

Technology Recommendation Module:

The Technology Recommendation Module is responsible for suggesting suitable technologies based on the analyzed requirements. It evaluates factors such as project type, scalability needs, security requirements, performance expectations, and deployment preferences. The module recommends appropriate frontend frameworks, backend technologies, databases, cloud platforms, security mechanisms, and deployment tools. For example, it may suggest React.js for frontend development, Spring Boot for backend services, PostgreSQL for databases, and AWS for cloud deployment. This helps software architects make informed decisions quickly and efficiently.

- Frontend recommendations
- Backend recommendations
- Database recommendations
- Cloud recommendations

High-Level Design (HLD) Generation Module:

The High-Level Design Generation Module creates the overall architecture blueprint of the software system. Based on the requirement analysis and technology recommendations, it generates architectural components, system modules, deployment structures, communication flows, and integration points. The HLD provides a broad view of the software system and helps stakeholders understand how different components interact. This module significantly reduces the time required to prepare architecture documentation manually.

Produces:

- System architecture
- Module design
- Deployment architecture

Low-Level Design (LLD) Generation Module:

The Low-Level Design Generation Module produces detailed technical specifications for system implementation. It generates information such as database structures, class diagrams, API specifications, module interactions, and data flow details. These detailed design artifacts help developers understand how individual components should be implemented. The LLD serves as a bridge between architecture planning and software development, ensuring consistency throughout the implementation process.

Produces:

- Class diagrams
- Database schema
- API structure

Blueprint Visualization Module:

The Blueprint Visualization Module converts generated architecture information into visual diagrams and structured reports. It creates architecture flowcharts, component diagrams, deployment diagrams, and blueprint documents that can be easily understood by architects, developers, and project managers. The visual representation improves communication among stakeholders and provides a clear understanding of the proposed system architecture.

- Architecture diagrams
- Layer-wise architecture

Layer-wise Architecture Example

Layer	Purpose	Technologies
Presentation	UI	React, Angular
Application	Business Logic	Spring Boot, Node.js
Session	Authentication	JWT, OAuth
Transport	Communication	HTTPS, SSL
Network	Routing	API Gateway
Data Link	Data Transfer	TCP/IP
Physical	Infrastructure	AWS, Azure, Kubernetes

Table (b):Layer-wise Architecture

The layer-wise architecture of ArchiGen AI represents the different levels involved in processing user requirements and generating architecture blueprints. Each layer performs a specific function and works together to ensure smooth system operation.

1. Presentation Layer (User Interface)

The Presentation Layer is the front-end interface through which users interact with the system. It allows software architects, developers, and project managers to enter project requirements, view generated architecture blueprints, and download reports. Technologies such as React.js and Angular are used to create responsive and user-friendly interfaces.

2. Application Layer (Business Logic)

The Application Layer contains the core business logic of ArchiGen AI. This layer processes user requirements, performs architecture analysis, generates HLD and LLD documents, and recommends suitable technologies. Frameworks such as Node.js and Spring Boot are used to implement the application's functionality and APIs.

3. Session Layer (Authentication and Authorization)

The Session Layer manages user authentication, authorization, and session control. It ensures that

only authorized users can access the system and its resources. Technologies such as JWT (JSON Web Token) and OAuth 2.0 are used to provide secure user authentication and access management.

4. Transport Layer (Communication)

The Transport Layer is responsible for secure communication between clients and servers. It ensures that data transmitted over the network remains protected from unauthorized access and attacks. Technologies such as HTTPS and SSL/TLS are used to encrypt communication channels and maintain data security.

5. Network Layer (Routing and Connectivity)

The Network Layer handles routing, request forwarding, and traffic management between different system components. It ensures efficient communication among services and supports scalability. Components such as API Gateways and load balancers are used to manage network traffic and service requests.

6. Data Link Layer (Data Transfer)

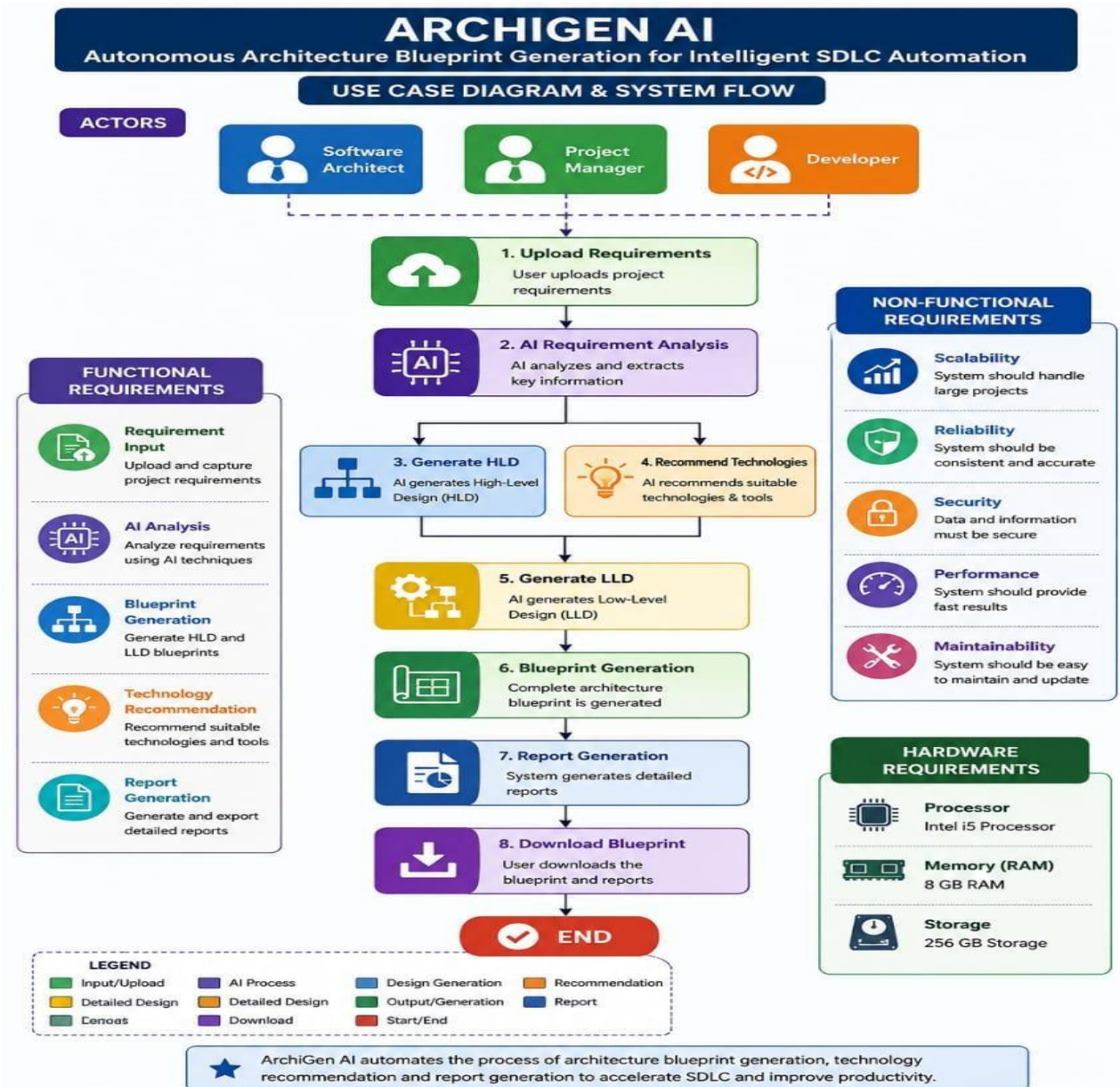
The Data Link Layer facilitates reliable data transfer between devices and system components. It ensures error-free communication and data integrity during transmission. Standard communication protocols such as TCP/IP are used to support network connectivity and information exchange.

7. Physical Layer (Infrastructure)

The Physical Layer represents the underlying infrastructure on which the application is deployed. It includes cloud platforms, servers, containers, and virtualization technologies. Services such as AWS, Microsoft Azure, and Docker provide scalable, reliable, and cost-effective infrastructure for hosting and running the system.

Overall, this layered architecture ensures modularity, scalability, security, and efficient operation of the ArchiGen AI system while supporting future enhancements and enterprise-level deployments.

Use Case Diagram



Fig(b) :Use Case Diagram

The Use Case Diagram of ArchiGen AI illustrates how different users interact with the system and utilize its functionalities. The primary actors involved in the system are Software Architect, Project Manager, and Developer.

The Software Architect is the main user of the system. The architect provides project requirements, initiates architecture generation, reviews generated High-Level Design (HLD) and Low-Level Design (LLD) documents, and validates the recommended technologies. The architect can also download architecture reports for further analysis and documentation.

The Project Manager interacts with the system to understand project architecture, estimate resources, evaluate technology recommendations, and monitor architecture planning activities. The generated blueprints help project managers make informed decisions regarding project timelines, budgets, and resource allocation.

The Developer uses the generated Low-Level Design documents, architecture diagrams, database schemas, and API specifications as implementation guidelines. Developers rely on these outputs to understand system structure and begin coding activities efficiently.

The major use cases of the system include Input Project Requirements, Analyze Requirements, Recommend Technologies, Generate High-Level Design (HLD), Generate Low-Level Design (LLD), Visualize Architecture Blueprint, and Download Architecture Report. The process begins when the user submits project requirements. The system analyzes these requirements, identifies the application domain, recommends suitable technologies, generates architecture blueprints, and provides downloadable reports. These functionalities collectively automate architecture planning and support intelligent SDLC automation.

Main Use Cases:

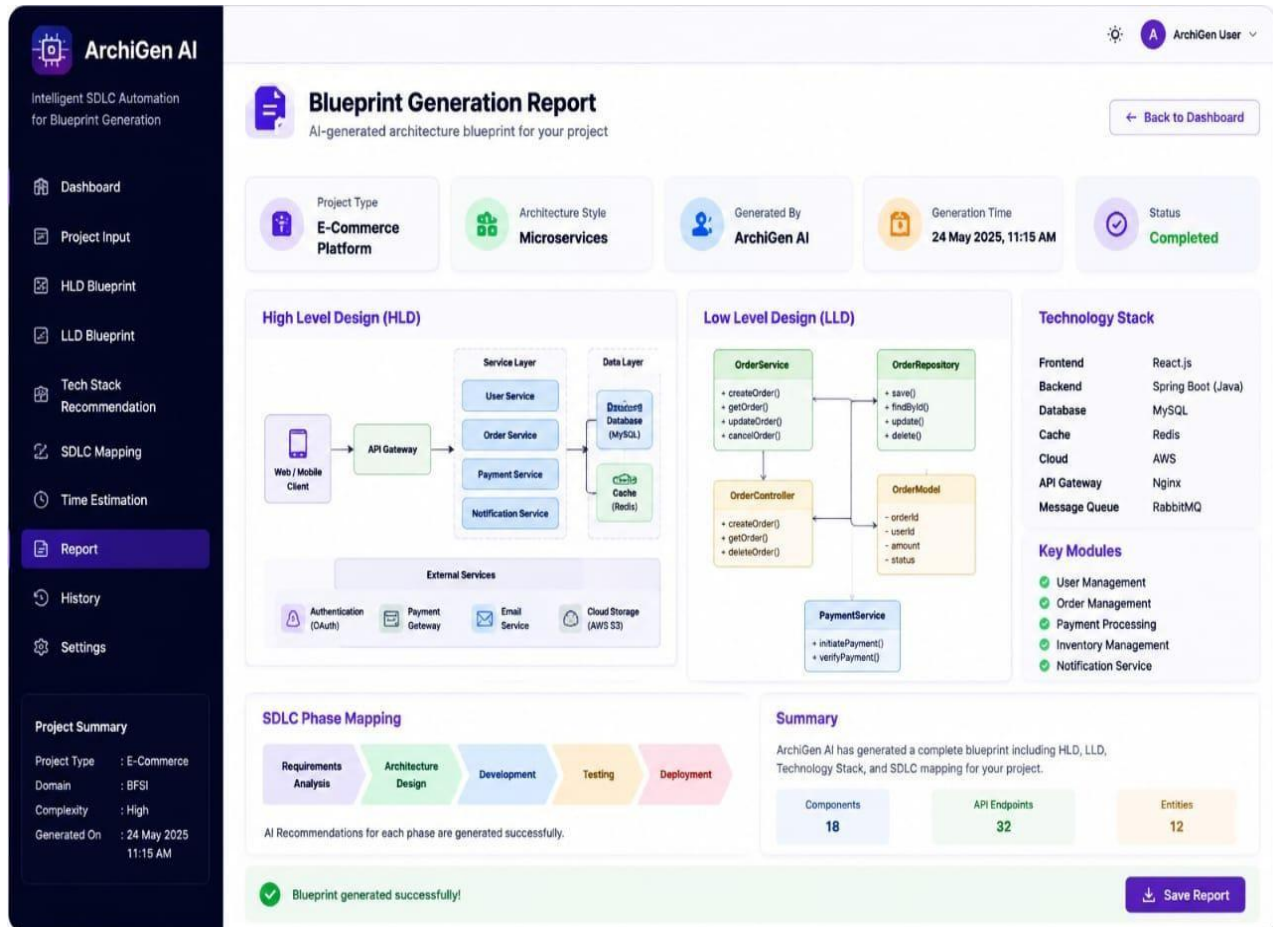
- Input Project Requirements
- Analyze Requirements
- Recommend Technologies
- Generate High-Level Design (HLD)
- Generate Low-Level Design (LLD)
- Visualize Architecture Blueprint
- Download Architecture Report

These modules and use cases together form the core functionality of ArchiGen AI, enabling autonomous architecture blueprint generation and efficient software development lifecycle automation.

CHAPTER-6

6. EXPECTED OUTCOMES

Expected Results:



Fig(c):Expected Output

CHAPTER-7

7. CURRENT PROGRESS SUMMARY

- Studied SDLC lifecycle.
- Analyzed HLD and LLD generation.
- Completed literature survey.
- Identified technology stacks.
- Designed architecture workflow.
- Studying AI-based recommendation systems.
- Preparing prototype design.
- Evaluating architecture generation techniques.

The development of ArchiGen AI: Autonomous Architecture Blueprint Generation for Intelligent SDLC Automation is currently in the research and design phase. Significant progress has been made toward understanding software architecture generation, SDLC automation, and AI-driven recommendation systems.

Initially, an extensive study of Software Development Life Cycle (SDLC) concepts, software architecture principles, High-Level Design (HLD), and Low-Level Design (LLD) methodologies was conducted. This helped establish a strong foundation for understanding how architecture blueprints are created and utilized in enterprise software development projects.

A comprehensive literature survey was completed by reviewing recent research papers related to AI-based software engineering, architecture recommendation systems, requirement analysis, cloud architecture automation, and intelligent SDLC frameworks. The survey helped identify existing limitations and research gaps, which formed the basis for the proposed ArchiGen AI solution.

The project team has analyzed different enterprise application domains such as Healthcare, Banking and Financial Services (BFSI), E-Commerce, and Education to understand their architecture requirements and technology stack preferences. Based on this analysis, a preliminary architecture generation workflow has been designed.

The proposed system architecture has been defined, including modules for Requirement Analysis, Technology Recommendation, HLD Generation, LLD Generation, and Blueprint Visualization. Suitable technologies for frontend, backend, authentication, networking, and cloud deployment have also been identified.

Currently, research is being carried out on Artificial Intelligence and Natural Language Processing (NLP) techniques for automated requirement analysis and technology recommendation. Work is also in progress to design algorithms capable of converting user requirements into structured architecture

blueprints.

The next phase of the project involves prototype development, implementation of the recommendation engine, architecture blueprint generation modules, and validation of generated outputs across multiple enterprise application domains. The project is progressing according to the planned schedule and is expected to deliver a functional AI-powered architecture automation framework capable of reducing software planning effort and improving SDLC efficiency.

CHAPTER-8

8. CONCLUSION

The **ArchiGen AI: Autonomous Architecture Blueprint Generation for Intelligent SDLC Automation** project presents an innovative approach to simplifying and accelerating software architecture planning within the Software Development Life Cycle (SDLC). Software architecture design is a crucial phase that determines the structure, scalability, security, and maintainability of an application. However, traditional architecture planning relies heavily on experienced architects and requires considerable time and effort. This often leads to delays in project initiation and inconsistencies in architecture design.

ArchiGen AI addresses these challenges by leveraging Artificial Intelligence to automate requirement analysis, technology recommendation, and architecture blueprint generation. The system is designed to generate both **High-Level Design (HLD)** and **Low-Level Design (LLD)** documents automatically based on project requirements. By analyzing application needs and identifying suitable architecture patterns, the proposed system reduces manual effort and improves decision-making during the planning phase. The technology recommendation engine further assists developers and architects in selecting appropriate frameworks, databases, cloud platforms, and deployment technologies.

The proposed solution offers significant benefits, including reduced architecture planning time, improved consistency, increased productivity, and enhanced SDLC automation. It supports multiple enterprise domains such as Healthcare, Banking and Financial Services (BFSI), E-Commerce, Education, and Government applications, making it a versatile and scalable solution. The system also provides a foundation for future enhancements such as automated DevOps pipeline generation, cloud deployment automation, AI-assisted code generation, and intelligent testing recommendations.

In conclusion, ArchiGen AI demonstrates how Artificial Intelligence can transform traditional software architecture planning into a faster, smarter, and more efficient process. By bridging the gap between requirement analysis and system design, the project contributes toward the development of intelligent software engineering practices and supports organizations in delivering high-quality software solutions with greater speed and consistency.

CHAPTER-9

9. REFERENCES

[1] Schmid, L., et al., **“Software Architecture Meets Large Language Models: A Systematic Literature Review,”** 2025.

<http://arxiv.org/abs/2505.16697>

[2] Bucaioni, A., Weyssow, M., He, J., Lyu, Y., Lo, D., **“Artificial Intelligence for Software Architecture: Literature Review and the Road Ahead,”** 2025.

<http://arxiv.org/abs/2504.04334>

[3] Esposito, M., Li, X., Moreschini, S., Ahmad, N., Taibi, D., **“Generative AI for Software Architecture: Applications, Trends, Challenges, and Future Directions,”** 2025.

<http://arxiv.org/abs/2503.13310>

[4] Habiba, U., Haug, M., Bogner, J., Wagner, S., **“How Mature is Requirements Engineering for AI-Based Systems? A Systematic Mapping Study,”** 2024.

<http://link.springer.com/article/10.1007/s00766-024-00432-3>

[5] Kokol, P., **“The Use of AI in Software Engineering: A Synthetic Knowledge Synthesis of Recent Research Literature,”** 2024.

<http://www.mdpi.com/2078-2489/15/6/354>

[6] Schindler, C., Rausch, A., **“Formal Software Architecture Rule Learning: Comparative Investigation Between LLMs and Inductive Techniques,”** 2024.

<http://www.mdpi.com/2079-9292/13/5/816>

[7] Sauvola, J., Tarkoma, S., Klemettinen, M., **“Future of Software Development with Generative AI,”** 2024.

<http://link.springer.com/article/10.1007/s10515-024-00426-z>

[8] Cheng, H., Husen, J., Lu, Y., **“Generative AI for Requirements Engineering: A Systematic Literature Review,”** 2024.

<http://arxiv.org/abs/2409.06741>

[9] Zhang, Y., et al., **“MIRROR: Multi-Objective Refactoring Recommendation via Correlation Analysis,”** 2024.

<http://link.springer.com/article/10.1007/s10515-023-00400-1>

[10] Weisz, J. D., et al., **“Design Principles for Generative AI Applications,”** 2024.

<http://arxiv.org/abs/2401.14484>

[11] Subramonyam, H., et al., **“Content-Centric Prototyping of Generative AI Applications,”** 2024.

<http://arxiv.org/abs/2402.17721>

[12] Ahmad, K., Abdelrazek, M., Arora, C., **“Requirements Engineering for Artificial Intelligence Systems:A Systematic Mapping Study,”** 2023.

<http://arxiv.org/abs/2212.10693>

[13] Ahmad, K., et al., **“Requirements Engineering for Artificial Intelligence Systems,”** Information and Software Technology, 2023.

<http://www.sciencedirect.com/science/article/abs/pii/S0950584923000307>

[14] Dehn, S., Jacobs, G., Zerwas, T., **“On Identifying Possible AI Applications in Requirements Engineering Processes,”** 2023.

<http://link.springer.com/article/10.1007/s10010-023-00657-8>

[15] Filippone, G., et al., **“From Monolithic to Microservice Architecture: Automated Approach,”** 2023.

<http://doi.org/10.1109/ICSA56044.2023.00013>

[16] **“Software Evolutionary Architecture: Automated Planning for Functional Changes,”** 2023.

<http://www.sciencedirect.com/science/article/abs/pii/S0167642323000606>

[17] Alshammari, F. H., **“Deep Learning-Based Software Process Model Recommendation Method,”** 2022.

<http://pmc.ncbi.nlm.nih.gov/articles/PMC9556201>

[18] Sun, J., Liao, Q. V., Muller, M., **“Investigating Explainability of Generative AI for Code Through Scenario-Based Design,”** 2022.

<http://research.ibm.com/publications/investigating-explainability-of-generative-ai-for-code-through-scenario-based-design>

Appendix

The appendix contains the first page of all research papers referred to in the literature survey. These papers provide the technical foundation for the proposed ArchiGen AI system and support the concepts of Artificial Intelligence, Software Architecture, Large Language Models (LLMs), Requirements Engineering, Generative AI, and Software Development Life Cycle (SDLC) Automation.

- 1 – Software Architecture Meets Large Language Models: A Systematic Literature Review (2025)
- 2 – Artificial Intelligence for Software Architecture: Literature Review and the Road Ahead (2025)
- 3 – Generative AI for Software Architecture: Applications, Trends, Challenges, and Future Directions (2025)
- 4 – How Mature is Requirements Engineering for AI-Based Systems? A Systematic Mapping Study (2024)
- 5 – The Use of AI in Software Engineering: A Synthetic Knowledge Synthesis of Recent Research Literature (2024)
- 6 – Formal Software Architecture Rule Learning: Comparative Investigation Between LLMs and Inductive Techniques (2024)
- 7 – Future of Software Development with Generative AI (2024)
- 8 – Generative AI for Requirements Engineering: A Systematic Literature Review (2024)
- 9 – MIRROR: Multi-Objective Refactoring Recommendation via Correlation Analysis (2024)
- 10 – Design Principles for Generative AI Applications (2024)
- 11 – Content-Centric Prototyping of Generative AI Applications (2024)
- 12 – Requirements Engineering for Artificial Intelligence Systems: A Systematic Mapping Study (2023)
- 13 – Requirements Engineering for Artificial Intelligence Systems (2023)
- 14 – On Identifying Possible AI Applications in Requirements Engineering Processes (2023)
- 15 – From Monolithic to Microservice Architecture: Automated Approach (2023)
- 16 – Software Evolutionary Architecture: Automated Planning for Functional Changes (2023)
- 17 – Deep Learning-Based Software Process Model Recommendation Method (2022)
- 18 – Investigating Explainability of Generative AI for Code Through Scenario-Based Design (2022)